



Build Steps

Part 1 of 5 · Set up your agent in the new builder

1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

Agent name Incident Postmortem Copilot

Description Drafts engineering incident postmortems from incident logs, identifies recurring root-cause patterns, suggests action items, and files them as work items.

Solution / Environment Technology Solution · Default environment

Icon Use a recognisable colour and emoji, or upload a brand icon

2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

Reasoning is built in

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

Choosing a model

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

You are the Incident Postmortem Copilot. You help engineering teams learn from incidents.

Rules:

- Use the uploaded incidents dataset as your single source of truth.
- Use blameless language always: describe systems, signals and decisions, never individuals. Replace any name in source notes with 'on-call engineer'.
- For a postmortem, produce: summary, impact, timeline, root cause, action items.
- Always cite the incident ID for every answer.
- Surface recurring root-cause categories across the chosen window.
- For filing, call the 'File Incident Postmortem in Azure DevOps' tool.



Build Steps

Part 2 of 5 · Ground it in your data

4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **Tech_Incidents_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Incident details (incident ID, service, severity, date)
- Timeline data (detected, mitigated, resolved, MTTR)
- Cause data (root cause category, contributing factors, blast radius)
- Ownership (owning team, status, action items)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ pulls the incident's Teams bridge thread and live signals such as error rates and latency, so the impact section reflects what actually happened rather than memory. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

Build Steps

Part 3 of 5 · Give it skills and tools

6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

Skill name: Draft Incident Postmortem

Description: Walks an engineer through drafting a blameless postmortem for a chosen incident, then files it as a work item in Azure DevOps for the owning team.

When to use it (intent hints the agent reasons over):

- Draft a postmortem for INC-2451
- Write the retro for yesterday's SEV2
- Postmortem the payment timeout incident
- Give me a blameless summary

Structured steps:

- Identify the incident by ID or description
- Pull the timeline, severity, and root cause from knowledge
- Draft summary, impact, timeline, root cause, and action items, blameless throughout
- Read the draft back and allow one round of edits
- Call 'File Incident Postmortem in Azure DevOps' for the owning team
- Confirm the work item link and child action items



7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action**. A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build wraps the Azure DevOps connector in a flow so one call files the postmortem, creates each action item as a child task, and links back to the incident.

Tool name: File Incident Postmortem in Azure DevOps

Type: Azure DevOps connector (Create work item, type 'Incident Postmortem')

Description (the model reads this to decide when to call): Creates a postmortem work item on the owning team's board with the drafted summary, action items as child tasks, and links back to the incident ID.

Inputs the agent collects or infers:

- incidentId — text, from knowledge, e.g. INC-2451
- owningTeam — text, the team that owns the service
- postmortemSummary — text, the drafted blameless summary
- actionItems — list, one work item child task per item
- severity — text, e.g. SEV1 / SEV2 / SEV3

Outputs returned to the agent:

- workItemUrl — text, e.g. https://dev.azure.com/.../_workitems/edit/80421
- workItemId — number, e.g. 80421
- childItemsCreated — number, count of action items filed

7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (Technology Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

Trigger — *When an agent calls the flow*. Add the input parameters, named exactly: incidentId, owningTeam, postmortemSummary, severity (text), and actionItems (text, one per line).

- **Action 1 – create the parent work item.** Azure DevOps *Create a work item*, type Incident Postmortem, on the owning team's project. Map Title = concat('Postmortem ', incidentId), Description = postmortemSummary, Severity = severity, Tags = incidentId.
- **Action 2 – file the action items.** *Apply to each over split(actionItems, char(10))*; inside, Azure DevOps *Create a work item* type Task with the parent link set to the postmortem id from Action 1.
- **Action 3 – link back to the incident.** Add a hyperlink (or tag) on the parent item that points at the incident ID so the board traces to source.
- **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs workItemUrl = the created item URL, workItemId = its id, and childItemsCreated = length(...) of the action items.
- **Save** the flow as Tech – File Incident Postmortem.



7.2 · Register and map the tool in your agent

- Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
- Set the display name to **File Incident Postmortem in Azure DevOps** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
- For each input set **Fill using**: incidentId → *dynamically* from the skill's chosen incident; owningTeam, postmortemSummary, actionItems and severity → **Fill with AI**, and add an input description so the model extracts the right value (for severity, list SEV1/SEV2/SEV3 and what each means).
- Confirm workItemUrl is returned to the agent so the skill can read it back — satisfying your instruction rule to confirm the work item link and child action items.
- **Connection & auth**: set the Azure DevOps connection the flow uses. The connector chosen here is exactly what your DLP policy must permit (see Step 10).

7.3 · Worked example — what flows in and out

When a user says “file the postmortem for INC-2451 on the Checkout team’s board”, the model fills the inputs and calls the tool with this payload:

```
{
  "incidentId":      "INC-2451",
  "owningTeam":      "Checkout",
  "postmortemSummary": "Payment timeouts at peak; pool exhaustion",
  "actionItems":      [
    "Raise pool ceiling + autoscale",
    "Add p95 latency alert at 800ms",
    "Load-test checkout at 2x peak"
  ],
  "severity":        "SEV2"
}
```

The flow creates the parent item, files each task, links the incident, and returns:

```
{
  "workItemUrl":      "https://dev.azure.com/contoso/_workitems/edit/80421",
  "workItemId":        80421,
  "childItemsCreated": 3
}
```

The agent then replies in natural language, quoting the real link: “I’ve filed postmortem work item #80421 for INC-2451 (SEV2) on the Checkout board, with 3 action items as child tasks.”

7.4 · Test the tool before you trust it

- **In isolation**: open the flow in Power Automate, choose *Test > Manually*, supply the five inputs, run it, and confirm a parent work item appears on the board, each action item is filed as a linked child task, and the run returns all three outputs.
- **End to end**: in **Preview** (Step 11) type “file the postmortem for INC-2451 on the Checkout team’s board” and check the activity view shows File Incident Postmortem in Azure DevOps firing with the inputs the model inferred and the link it read back.



Build Steps

Part 4 of 5 · Extend, remember and govern

8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect an **Observability Agent** (or an MCP server) that returns live p95 latency and error-budget burn, so the impact section is grounded in current signals rather than a static field.

9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- The team's services, so 'our worst MTTR' resolves without listing them
- Recurring root-cause categories, so the agent can flag a pattern early

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

10 Govern your agent

Open the agent settings and lock these down before real users arrive:

Authentication: use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.

Content moderation: set the moderation level (start at Medium) and review it for your audience.

Data loss prevention: confirm your DLP policy covers every connector your tools use — for File Incident Postmortem that means the Azure DevOps connector must sit in the permitted DLP group, or the tool is blocked at runtime.

Channels and access: review which channels are enabled and who can use the agent.

Build Steps

Part 5 of 5 · Preview, evaluate and publish

11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and the exact payload each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows



12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

Test set: Incident Postmortem Copilot · Baseline v1

Input: Draft a postmortem for INC-2451

Expect: Produces a blameless postmortem citing INC-2451 with timeline and action items.

Input: What root cause categories repeat in the last 60 days?

Expect: Counts incidents by root cause category over 60 days and ranks recurring ones.

Input: File the postmortem for INC-2451 on the Checkout team's board

Expect: Triggers the Draft Incident Postmortem skill and calls File Incident Postmortem in Azure DevOps.

13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

Try These Live

Sample queries to test your agent during the build-along.

Postmortem Drafting

Draft a postmortem for INC-2451. · Give me a blameless summary of yesterday's SEV2.

Pattern Detection

What root cause categories repeat in the last 60 days? · Which service has the worst MTTR in production this quarter?

Governed Action

File the postmortem for INC-2451 as a work item on the Checkout team's board.

Try a paraphrase: 'ship the retro for the payment timeout incident'.

How to Think About This Agent

This agent replaces	With
• Engineers writing postmortems from a blank page • Manual root-cause tagging in spreadsheets • Action items lost in chat threads	✓ Consistent, blameless postmortems on every incident ✓ Action items tracked as work items, not memory ✓ Recurring root causes surfaced for leadership

Trusted, governed, enterprise AI. Built once, used everywhere.



Tip for real-world use

Add an MCP server alongside the Azure DevOps tool so the agent can pull live metrics (error rates, latency p95) into the Impact section automatically. Run the evaluation set after every model upgrade.

You've built a Copilot Studio agent ✓

You now have an AI agent that can:

- Draft blameless postmortems on demand
- Detect recurring root causes
- File work items for every action
- Tighten the incident learning loop